

Entity Resolution with FAISS + Splink

One system, three conversations

Denis Robert

How to use this deck

Part	Audience	Focus	After the talk
1	C-suite	Value, risk, cost	Decision / review
2	Engineering leadership	Architecture, design, roadmap	Carry it forward
3	Line engineers	Orientation + map to the paper	Read the paper

Line engineers: this orients you. The whitepaper (`docs/entity_resolution_whitepaper.pdf`) is the source of record; the numbers here come from it.

Part 1 · C-Suite

The 60-second version

The problem

Organisations hold **many records for the same real person**, and they disagree.

- Names differ (e.g., **R. Martinez** vs **Robert Martinez**).
- Addresses change; emails and dates go missing (**30% missing** here).
- Duplicates fragment CRM, risk, and reporting.

Wrong answers are expensive: linking two *different* people is usually far worse than missing two records of the same person.

The solution: two stages

Fast retrieval + explainable matching.

```
Query → [embed] → [FAISS top-k]  
      → [Splink linkage] → Match / No match
```

- **Blocking** (FAISS): cheaply finds the few most likely matches.
- **Linkage** (Splink): weighs the evidence per field and returns a probability, not just yes/no.

What it delivers

- **99.5% F1** on the synthetic test corpus (15,000 labelled queries).
- **~7 ms per query** — sub-millisecond blocking, probabilistic verdict on top.
- **Persisted and reusable**: the index reloads without re-embedding the population.
- **Defensible numbers**: every result reproduces by re-running the code.

Cost and scale

- The 50,000-record index is ~87 MB today.
- Capacity is predictable from RAM;

RAM	Recommended capacity
16 GB	~6.9 M records
32 GB	~13.8 M records
128 GB	~55.1 M records

Around **7–14 M records** (or when you need replicated, multi-region durability) buy a **vector database**; before that the in-process index is the lean choice.

The honest caveats

- Results are measured on **synthetic data**; production needs a **labelled sample of true duplicates** to calibrate and validate.
- **Retraining the match model without also resetting the threshold backfires.** The shipped defaults work best until proper joint calibration. Don't let a team "just retune" today.
- Volume must clear our capacity thresholds before any external-infra spend.

The ask

- Sponsor a short **proof-on-real-data** phase: a representative, labelled sample of the population we must link (with access/privacy).
- We then commit to an F1 on *that* data, not the synthetic one.

Value in one line: near-perfect, explainable identity resolution at ~7 ms, with a priceable path to scale.

Part 2 · Engineering Leadership

Architecture, measurements, where to invest

Architecture at a glance

Query $\longrightarrow$ MiniLM embed $\longrightarrow$ FAISS top-k $\longrightarrow$ Splink $\longrightarrow$ $p(M \mid \gamma)$

▲
name / DOB / email / address comparisons, τ threshold

- Embeddings are **L2-normalized**, so FAISS inner product = cosine.
- Splink = **Fellegi–Sunter**: per-field weight $w = \log_2(m/u)$, combined with a match prior, compared to τ .
- A store abstraction lets us swap the in-memory index for an **external vector DB** later without changing blocking or linkage.

Where performance comes from

Blocking recall is the ceiling; Splink is where F1 is won or lost.

- Top-20 blocking recall: **99.84%** (default), **99.95%** (compact serialization).
- End-to-end recall: **99.43%** → the ~0.4% gap is lost in *linkage*, not blocking.
- A "perfect" blocker buys only ~**0.08% more F1** (99.66 → 99.74% ceiling).

Design implication: improving the embedding/blocking engine is **not** the lever today; linkage configuration is.

The measured levers

Lever	Effect	Verdict
Threshold τ	0.85→0.95 lifts F1 99.53→99.63%	Yes — cheap precision knob
Serialization	compact: recall 99.95%, F1 99.77%	Yes — small but real
Blocking size k	20→100: +recall, 2.4× Splink time	Diminishing returns
Weaken address	never beat full-strength	No (this data)
Retrain m/u	untrained F1 0.997 vs trained ≤ 0.974	Keep defaults

The calibration paradox

Retraining the match model (m/u) made results **worse**, not better.

1. **Prior coupling (dominant):** EM's fitted prior (0.0072 vs 0.0001) shifts all scores up → mass false positives. Pinning the prior raises EM's F1 from ~0.86 to ~0.95–0.97.
2. **A genuine residual:** even with the prior pinned, trained m/u stay below defaults (≤ 0.974 vs 0.997) by over-weighting partial matches.

Rule: the default config is a coherent bundle (weights + prior + τ). Change any part and re-validate **all three** on held-out data.

See paper §8.1 and the joint $\tau \times \text{prior}$ table.

Capacity and when to buy infra

- **Memory is the binding constraint** (~1,746 bytes/record).
- Move to an **external vector DB** around **7–14 M records**, or when you need shared/multi-region **durability and replication**, or beyond **~50 M** as a rule of thumb.
- The persisted index reloads **without re-embedding**, so redeploy and multi-consumer serving are cheap.

Operational readiness

- Persisted, reload-friendly store with `update` / `delete` ; external-store interface is the vector-DB swap point.
- **Reproducibility built in:** every experiment maps to a script; scripts accept `--input` to run on other datasets.

Roadmap

1. Invest in **linkage calibration** (labelled pairs + joint τ /prior tuning).
2. Do **not** spend on blocking-engine research at this scale.
3. Add a **validation/CI gate** that locks F1 on a held-out labelled set.
4. Drive the NC-voter / real-data replication to a decision gate.

Part 3 · Line Engineers

Orientation → then go read the paper

What the code does

```
Blocker    : embed query → FAISS top-k candidates (cosine)
Linker     : Splink comparisons → per-candidate match probability
Store      : add / update / delete / save / load (no re-embed on load)
```

- Match probability = Bayesian combination of per-field weights $w = \log_2(m/u)$, a match prior, thresholded at τ .
- In-memory index today; `IndexingStrategy` / `VectorDatabase` is the seam for a vector DB later.

Scorecard to remember

- Confusion matrix (5k refs / 15k queries): **F1 99.45%**, recall 99.43%, precision 99.48%.
- Blocking recall: **99.84%** @k=20 (99.95% compact).
- **7.25 ms/query** (embed + block + batched Splink).
- NC-voter (real, mutated): **F1 92–94%**; blocking is the binding constraint.

Baseline engineering numbers — config, hardware, and data shape all move them.

Reproducing experiments

Run from repo root; scripts have `--help` and deterministic seeds (default 42).

Paper section	Script
§7 evaluation	<code>scripts/experiment_section7_eval.py</code>
§8 confusion matrix	<code>scripts/experiment_confusion_matrix.py</code>
§8.1 m/u calibration	<code>scripts/experiment_mu_calibration.py</code>
§8.2 duplicate benchmark	<code>scripts/experiment_duplicate_benchmark.py</code>
§8.3 F1 sweep	<code>scripts/experiment_f1_sweep.py</code>
calibration paradox	<code>scripts/experiment_mu_tau_interaction.py</code>
joint $\tau \times$ prior surface	<code>scripts/experiment_mu_prior_tau_surface.py</code>
NC-voter replication	<code>scripts/ncvoter/*</code> + <code>extract_ncvoter.py</code>

For your data: population-based scripts accept `--input-records FILE`.

Gotchas you will hit

- Don't "just retune" m/u . It looks worse (prior/ τ coupling). Re-validate prior + τ + weights together.
- **Blocking recall is a hard ceiling.** If recall is low, raise k or serialization, not the embedding model.
- **Address weakening didn't help** here — test, don't assume.
- **Real data needs a mutation/duplicate model** to score (`scripts/ncvoter/ncvoter_util.py`).
- NC voter has **no full DOB and no email** — those comparisons are weak there.

Now read the paper

Whitepaper: `docs/entity_resolution_whitepaper.pdf` (source: `.tex`).

- **§3 Two-Stage** — blocking/linkage contract and costs.
- **§7 Evaluation** — recall, thresholds, latency, ablations.
- **§8, 8.1, 8.2, 8.3** — headline results and the "don't retune alone" evidence.
- **§9 NC-voter** — real schema, mutation model, `k` scaling.
- **Appendix: Deriving τ** — cost / F1 / GMM / transitivity methods.
- **Use of AI** — disclosure and verification.

Checklist before you leave

1. Run `scripts/experiment_confusion_matrix.py --count 5000` and read the JSON.
2. Reproduce the paradox: `scripts/experiment_mu_tau_interaction.py --base-count 5000`.
3. Swap `--input-records <your file>` into a population-based experiment.
4. Find where **blocking recall** caps your F1 — then plan linkage work.

That is the fastest way to make the paper your own.

Thank you

FAISS + Splink — one pipeline, three audiences

Bring a labelled sample of your real duplicates.